用 AI Skills 打通中间件迁移：定位服务从 Android 到鸿蒙的完整实践

原创 用户&内容技术 大淘宝技术 2026年6月14日 23:06 浙江 8人



本文以Android到鸿蒙的定位服务迁移为实战案例，深入剖析了AI辅助开发中通用智能与领域知识断层的根本矛盾，提出并验证了“AI + Skills”解决方案。该方案通过将API映射、枚举细节、回调差异及常见陷阱等隐性知识转化为结构化、AI可读的Skills文档，明确了AI负责通用逻辑生成、Skills提供精准领域知识的分工模型，实现了从“面向人”到“面向AI”的知识传递转变。实践表明，相比纯AI翻译的低准确率和人工查源码的低效率，AI + Skills模式不仅将单服务迁移时间缩短至30分钟且零编译错误，更在154个服务的规模化迁移中节省25小时，实现了知识的资产化、可复用与持续演进，最终展望了从静态文档向知识图谱、主动建议及组织级知识中台发展的未来路径。

01 引言：AI 辅助开发的根本矛盾

> 又双叒叕经历从0到1翻译一个安卓应用到鸿蒙的需求，并且近期Skill的概念很火，想着能通过Skill真正的解决一些实际业务遇到的一些问题，所以开始着手维护一个鸿蒙依赖分析的Skill，为了协助Curosr更好的理解端侧的二方包，最后我得出了两个暴论：
> 1、目前很多知识库是冗余的，随着模型能力越来越强，需要提供的业务知识库是需要减负的
> 2、未来我们各种二方包的接入方式都会以Skill的形式提供，从面向人转到面向AI

现状：AI 很强大，但不够"准确"

场景还原：

```
// 你向 AI 提问："帮我将 Android 定位服务迁移到鸿蒙"
// AI 生成的代码（看起来很专业）
LocationExpires.ONE_MINUTE     // ❌ 这个枚举值不存在
params.onSuccess = (result) => { ... }  // ❌ 回调位置错误
// 编译器提示：13 个错误
```

```
6  Property 'ONE_MINUTE' does not exist on type 'LocationExpires'
```

开发者的困惑：

- AI 不是能理解代码吗？为什么会犯这种低级错误？
- 文档明明存在，为什么 AI 不看文档？
- 第一次踩坑修好了,第二次还会踩,知识怎么沉淀？

本质：AI 的能力边界

AI 能做的（通用能力）	AI 不能做的（领域知识）
✅ 理解代码结构	❌ 精确的枚举值（`ONE_MIN` vs `ONE_MINUTE`）
✅ 语义转换	❌ API 调用细节（回调在哪个参数上）
✅ 生成代码框架	❌ 平台特性（隐私合规、权限系统）
✅ 推理和优化	❌ 实时 API 变化（新版本的枚举值）

根本矛盾：

```
AI 的通用智能 ≠ 领域专家知识
        ↓
知识断层 → 低质量输出 → 反复调试 → 知识无法沉淀
```



这不只是"代码翻译"问题

表面问题：Android 代码如何迁移到鸿蒙？

深层问题：

1. **知识获取**：开发者如何快速掌握新平台的 API？
2. **知识传递**：团队如何避免重复踩坑？
3. **知识演进**：API 更新后如何快速同步？
4. **AI 协同**：如何让 AI 使用最新的领域知识？

对比传统方式：

场景	传统方式	痛点	AI + Skills 方式
新人学习	看文档 + 问老人	文档滞后、信息分散、重复提问	AI 读 Skill → 5 分钟上手
代码迁移	人工翻译	2-3h/服务、易出错、不可复用	AI 生成 → 30 分钟完成
知识更新	口口相传	经验流失、知识孤岛	更新 Skill → 全员同步
质量保证	Code Review	依赖人工、滞后发现	AI 编译前检查 → 0 错误

这是一个"知识工程"问题

软件工程的两大知识库：

```
通用知识（AI 擅长）                领域知识（AI 不擅长）
├── 编程语言语法                   ├── 公司内部框架的枚举值
├── 设计模式                       ├── 第三方 SDK 的调用细节
├── 算法                           ├── 平台特性和限制
└── 架构模式                       └── 团队最佳实践
        ↑                                  ↑
    ChatGPT                          需要 Skills
```

知识的三个状态：

1. **隐性知识：**在老员工脑子里（"这里要用 ONE_MIN，别用 ONE_MINUTE"）
2. **显性知识：**在文档里（但分散、滞后、难搜索）
3. **可执行知识：**在 Skills 里（结构化、可索引、AI 可读）

AI + Skills 的价值：

```
隐性知识 → Skills 结构化 → AI 索引 → 精准输出
    ↓
知识资产化、可复用、可演进
```

03 AI + Skills 方法论

核心概念

定义：

AI + Skills = AI 的通用能力 + 可执行的领域知识包

类比理解：

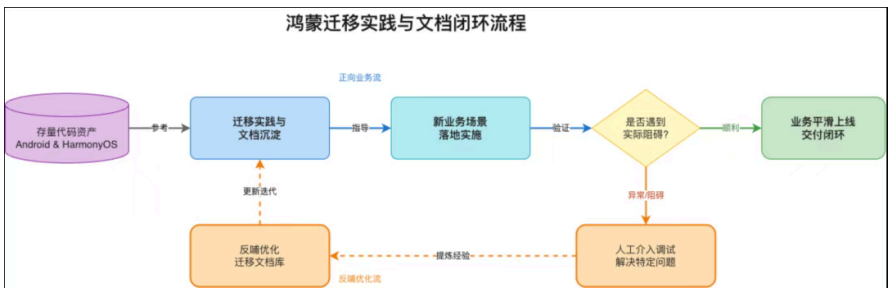
类比	AI	Skills	协同效果
医生看病	通用医学知识	本院设备使用手册	快速确诊 + 精准治疗
律师辩护	法律条文	本地判例库	通用法律 + 地方经验
AI 开发	编程能力	公司内部框架文档	通用代码 + 准确 API

分工模型

```
┌──────────────────────────────────────────────────────────┐
│                        开发任务                          │
└──────────────────────────────────────────────────────────┘
                            ↓
┌──────────────────────────────────────────────────────────┐
│  AI 负责（通用能力）          Skills 负责（领域知识）     │
│  ├── 理解业务逻辑             ├── API 映射表             │
│  ├── 生成代码结构             ├── 枚举值完整列表         │
│  ├── 语义转换                 ├── 平台特性说明           │
│  └── 代码优化                 └── 常见陷阱记录           │
└──────────────────────────────────────────────────────────┘
                            ↓
┌──────────────────────────────────────────────────────────┐
│              AI 查询 Skills → 生成精准代码               │
└──────────────────────────────────────────────────────────┘
                            ↓
┌──────────────────────────────────────────────────────────┐
│          编译通过 → 沉淀经验 → 更新 Skills               │
└──────────────────────────────────────────────────────────┘
```

工作流程

完整闭环：



关键环节：

1. **输入阶段：** AI 加载 Skills 获取领域知识
2. **生成阶段：** AI 使用 Skills 中的映射表
3. **验证阶段：** 编译/测试验证准确性
4. **反馈阶段：** 问题 → 提炼 → 更新 Skills
5. **沉淀阶段：** 成功经验 → 记录到 Skills



实战案例 安卓定位服务 – 鸿蒙代码迁移

问题背景

业务场景： 穿搭业务需要从0到1迁移到鸿蒙

技术挑战：

- 154 个 Android 服务需要迁移
- 每个服务涉及不同的 API 映射
- 团队中大部分人不熟悉鸿蒙 API

传统方式的成本：

```
1 个服务 × 2-3 小时 × 154 个服务 = 308-462 小时（13-19 天）
                                        ↓
                                知识无法复用，下次还要重新学
```

三种方式对比

- 方式 1：纯 AI 翻译（❌ 快但不准）

操作：

提问："帮我将 Android 的 LBSService 翻译到鸿蒙"

AI 生成的代码：

```
// AI 基于"常识"猜测的枚举值
LocationExpires.ONE_MINUTE     // ❌ 实际是 ONE_MIN
LocationExpires.FIVE_MINUTE    // ❌ 实际是 FIR_MIN
```

```
LocationAccuracy.MID_MODE     // ❌ 这个枚举不存在
// AI 推测的回调方式
const params = new LocationRequestLocationParams();
params.onSuccess = (result) => { ... };  // ❌ 回调不在这里
```

结果：

- 编译错误：**13 个**
- 调试时间：无法估计
- 根本原因：AI 没有准确的 API 文档

- 方式 2：查源码 + 人工修正（✅ 准但慢）

操作流程：

1. 打开 Mega Location 源码
2. 查看实际的枚举定义
3. 逐个修正 AI 生成的错误
4. 测试验证

发现的真相：

```
// ✅ 实际的枚举定义（非常反直觉）
export enum LocationExpires {
  ONE_MIN = "ONE_MIN",      // 不是 ONE_MINUTE
  SEC_MIN = "SEC_MIN",      // 不是 TWO_MINUTE (SEC = SECOND)
  THR_MIN = "THR_MIN",      // 不是 THREE_MINUTE (THR = THREE)
  FOR_MIN = "FOR_MIN",      // 不是 FOUR_MINUTE (FOR = FOUR)
  FIR_MIN = "FIR_MIN"       // 不是 FIVE_MINUTE (FIR = FIVE)
}
// ✅ 实际的回调方式
const options: Location.LocationRequestOptions = {
  onSuccess: (result: LocationData) => { ... }  // 回调在 options 内
};
```

结果：

- 编译错误：**0 个**

- 耗时：**40 分钟**
- 问题：知识留在开发者脑子里，下次还要重新查

- 方式 3：AI + Skills（✅ 又快又准）

Step 1：构建 Skills

```
.cursor/skills/鸿蒙依赖适配分析/
├── SKILL.md
└── reference/                              # 参考资料目录
    ├── android-to-harmony-mapping.md       # Android 到鸿蒙依赖映射表
    ├── dependency-analysis-guide.md        # 依赖分析指南
    └── middleware-docs/                    # 中间件文档目录
        ├── location-usage.md               # 定位服务使用文档 ✅ （此处用的这个
        ├── image-usage.md                  # 图片加载使用文档
        ├── login-usage.md                  # 登录服务使用文档
        ├── mtop-usage.md                   # Mtop 网络使用文档
        ├── ....
```

```
## 4. API 对比
### 4.1 LocationExpires 枚举值映射
| Android 常量 | 鸿蒙枚举 | 说明 |
|--------------|---------|------|
| "1m" | LocationExpires.ONE_MIN | ⚠️ 不是 ONE_MINUTE |
| "2m" | LocationExpires.SEC_MIN | SEC = SECOND，不是 TWO_MIN |
| "3m" | LocationExpires.THR_MIN | THR = THREE，不是 THREE_MIN |
| "4m" | LocationExpires.FOR_MIN | FOR = FOUR，不是 FOUR_MIN |
| "5m" | LocationExpires.FIR_MIN | FIR = FIVE，不是 FIVE_MIN |
⚠️ **不存在的枚举值**（AI 经常错误生成）：
- ❌ `ONE_MINUTE`, `TWO_MINUTE`, `FIVE_MINUTE`
- ❌ `TEN_MINUTE`, `MID_MODE`
### 4.2 定位请求方法对比
| Android | 鸿蒙 | 差异说明 |
|---------|------|---------|
| `LocationServiceBridge.requestLocation()` | `Location.requestLocatio
#### 关键差异：回调设置方式
**Android**（回调作为独立参数）：
```kotlin
LocationServiceBridge.requestLocation(
    params,
    { result -> ... },  // 成功回调
    { error -> ... }    // 失败回调
)
```
```

鸿蒙（回调在 options 对象内）：

```
const options: Location.LocationRequestOptions = {
  bizName: 'TB_SHOPPING_PROCESS',
  onSuccess: (result: LocationData) => { ... },  // ✅ 在 options 内
  onFail: (error: string) => { ... }
};
Location.requestLocation(params, options);
```

三种方式的本质差异

对比维度	纯 AI	人工查源码	AI + Skills	本质差异
知识来源	AI 猜测	人脑记忆	Skills 文档	**知识载体**
准确性	低（13 错误）	高	高	**信息质量**
效率	快（但要调试）	慢（40 分钟）	快（0 调试）	**时间成本**
可复用	❌	❌	✅	**资产沉淀**
可演进	❌	❌	✅	**知识更新**
可传播	❌	❌	✅	**团队协作**

关键洞察：

```
纯 AI：通用智能，缺少领域知识 → 猜测 → 出错
人工：有领域知识，但在大脑里 → 不可复用
Skills：将领域知识结构化 → AI 可读 → 精准输出 → 知识资产化
```

规模化效果

单个服务的迁移效率：

方式	首次迁移	第 2 次	第 10 次	第 100 次	学习曲线
人工查源码	40 分钟	40 分钟	40 分钟	40 分钟	每次重新学
AI + Skills	30 分钟	30 分钟	30 分钟	30 分钟	0 学习成本

154 个服务的总成本：

方式	总时间	知识沉淀	新人学习	知识更新
人工查源码	**102 小时**	❌ 人脑	需要师徒传承	口口相传
AI + Skills	**77 小时**	✅ Skills	AI 5 分钟教会	更新文档即同步

节省时间： `102 - 77 = 25 小时`

关键价值：

- ⏰ **效率提升**：节省 25 小时
- 📚 **知识资产**：154 个服务的迁移经验永久沉淀
- 🚀 **团队加速**：新人 0 学习成本
- 🔄 **持续演进**：API 更新后，只需更新 Skills



适用场景

AI + Skills 模式不只适用于代码迁移，而是适用于所有**需要领域知识的 AI 辅助开发场景**：

场景	传统痛点	AI + Skills 解法
新技术学习	文档分散、学习曲线陡峭	Skills 提炼核心知识 → AI 5 分钟教会
代码迁移	人工翻译慢、易出错	Skills 提供映射表 → AI 自动翻译
框架升级	API 变更后全员踩坑	更新 Skills → AI 使用新 API
最佳实践	口口相传、知识流失	Skills 记录陷阱 → AI 自动规避
Code Review	依赖人工、发现滞后	Skills 定义规范 → AI 编译前检查
新人培训	师徒传承、效率低	AI 读 Skills → 即时答疑

通用模式：

`有明确知识的场景 → 知识结构化为 Skills → AI 索引使用 → 提升质量和效率`

构建 Skills 的原则

- 原则 1：AI 友好的结构化

❌ **不友好的文档**（人类可读，AI 难索引）：

```
定位服务的枚举值有很多，比如 ONE_MIN 代表 1 分钟，
注意不要写成 ONE_MINUTE，这是不存在的...
```

✅ **AI 友好的文档**（表格化、结构化）：

```
| Android | 鸿蒙 | 说明 |
|---------|------|------|
| "1m" | LocationExpires.ONE_MIN | ⚠️ 不是 ONE_MINUTE |
```

- 原则 2：持续演进

Skills 更新流程：

遇到问题 → 查看源码 → 提炼规律 → 更新 Skills → 发布版本 → 团队同步

- 原则 3：分层组织

Skill 文档结构：

```
middleware-docs/
├── location-usage.md       # 定位服务
├── network-usage.md        # 网络服务
├── storage-usage.md        # 存储服务
└── common-patterns.md      # 通用模式

每个文档包含：
├── 1. 依赖信息
├── 2. 初始化配置
├── 3. API 对比（⭐ 核心）
├── 4. 使用示例
├── 5. 常见问题
├── 6. 迁移注意事项
├── 7. 常见陷阱（⭐ 核心）
└── 8. 代码翻译模板
```



从 Skills 到知识图谱

当前：文档化的 Skills

```
location-usage.md
network-usage.md
storage-usage.md
```

未来：知识图谱

```
LocationService
  ├── 依赖于 → PrivacyService
  ├── 被调用于 → ShoppingService
  ├── API 映射 → LocationExpires 枚举
  └── 常见陷阱 → 回调位置错误
```

价值：

- AI 可以理解模块间的依赖关系
- 自动推荐相关的 Skills
- 提供跨模块的最佳实践

从被动查询到主动建议

当前：开发者主动询问 AI

```
开发者："帮我迁移定位服务"
AI：读取 location-usage.md → 生成代码
```

未来：AI 主动发现问题

```
AI 检测到：你正在使用 LocationExpires.ONE_MINUTE
AI 提示：这个枚举不存在，是否改为 ONE_MIN？
```

技术方案：

- IDE 插件实时分析代码
- 匹配 Skills 中的常见陷阱
- 编译前预警

从静态文档到动态生成

当前：人工编写 Skills

`工程师踩坑 → 查源码 → 编写文档 → 更新 Skills`

未来：AI 自动生成 Skills

```
AI 分析：
  - 源码变更记录
  - 编译错误日志
  - 团队讨论记录
  ↓
自动生成/更新 Skills
```

价值：

- Skills 永远是最新的
- 0 人工维护成本
- 覆盖所有场景

从个人工具到组织能力

当前：个人/团队使用

开发者 → AI + Skills → 提升个人效率

未来：组织级知识平台

```
┌──────────────────────────────────────────┐
|          组织知识中台                     |
|   ├── 技术 Skills（框架、API）            |
|   ├── 业务 Skills（领域知识）             |
|   ├── 流程 Skills（开发规范）             |
|   └── 文化 Skills（团队最佳实践）         |
└──────────────────────────────────────────┘
              ↓
    AI 助手 + 开发者
              ↓
     组织能力外化、可传承
```

终极愿景：

- 新人入职第一天就能高效工作（AI 教学）
- 老人离职后知识不流失（Skills 沉淀）
- 组织智慧持续积累（每次踩坑都是资产）

07 结语：重新定义软件工程的知识传递

传统模式的困境

```
隐性知识（老员工脑子里）→ 口口相传 → 知识流失
        ↓
   重复踩坑 → 效率低下 → 人才依赖
```

AI + Skills 的突破

```
隐性知识 → Skills 结构化 → AI 索引 → 精准输出
        ↓
   知识资产化 → 可复用 → 可演进 → 组织能力提升
```

核心价值主张

1. **效率提升：** AI 的速度 + 领域专家的准确度
2. **知识沉淀：** 从"人脑记忆"到"组织资产"
3. **持续演进：** 每次踩坑都让 Skills 更完善
4. **团队赋能：** 新人 0 学习成本，老人不再重复劳动



本文作者开益，来自淘天集团–用户终端技术团队。我们服务于淘宝基础用户产品，是淘宝最重要的业务线之一。首页、信息流推荐、消息聊天、搜索、我淘、用增、互动、内容等亿级用户规模产品，为我们带来大量业务/技术挑战及机会。

团队在保证业务的同时，以先进的跨端框架和研发模式不断完善自己，打造最极致的体验和工程技术，保障多端设备的适配和稳定运行，并探索端智能等创新机会，通过技术高效驱动业务的良性发展。持续探索以AI为底座构建从需求到上线的端到端自动化与产品化能力，使亿级规模的交付更快、更稳、更可控。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 2554

留言

写留言

还没有留言，来写首评